

4.1.2. Dictionary to dataframe

22:40

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35],
```

```
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
```

```
new_name = input("New name: ")
```

```
new_age = int(input("New age: "))
```

```
df.loc[len(df)] = [new_name, new_age]
```

```
# Display the DataFrame after adding a new row
```

```
print("After adding a row:\n", df)
```

```
# Modifying a row

row_to_modify = int(input("Index of row to modify: "))

new_age_value=int(input("New age: "))

df.at[row_to_modify,"Age"]=new_age_value


# Display the DataFrame after modifying a row

print("After modifying a row:")

print(df)


# Deleting a row

row_to_delete = int(input("Index of row to delete: "))

df=df.drop(index=row_to_delete).reset_index(drop=True)

# Display the DataFrame after deleting a row

print("After deleting a row:")

print(df)


# Adding a new column

genders=input("Enter genders separated by space: ").split()

df["Gender"]=genders


# Display the DataFrame after adding a new column

print("After adding a new column:")

print(df)
```

```
# Modifying a column
```

```
df["Name"]=df["Name"].str.upper()
```

```
# Display the DataFrame after modifying a column
```

```
print("After modifying a column:")
```

```
print(df)
```

```
# Deleting a column
```

```
df=df.drop(columns=["Age"])
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

Average time
0.466 s
465.50 ms



Maximum time
0.466 s
466.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ 1 out of 1 hidden test case(s) passed

✓ Test case 1 465 ms Debug

Expected output

Actual output

Original DataFrame:

Original DataFrame:

.....Name..Age

.....Name..Age

0.....Alice...25

0.....Alice...25

1.....Bob...30

1.....Bob...30

2..Charlie...35

2..Charlie...35

New name: Susan

New name: Susan

New age: 29

New age: 29

After adding a row:

After adding a row:

.....Name..Age

.....Name..Age

0.....Alice...25

0.....Alice...25

1.....Bob...30

1.....Bob...30

2..Charlie...35

2..Charlie...35

3...Susan...29

3...Susan...29

Index of row to modify: 0

Index of row to modify: 0

New age: 27

New age: 27

After modifying a row:

After modifying a row:

.....Name..Age

.....Name..Age

0.....Alice...27

0.....Alice...27

1.....Bob...30

1.....Bob...30

2..Charlie...35

2..Charlie...35

3...Susan...29

3...Susan...29

Index of row to delete: 3

Index of row to delete: 3

After deleting a row:

After deleting a row:

.....Name..Age

.....Name..Age

0.....Alice...27

0.....Alice...27

1.....Bob...30

1.....Bob...30

2..Charlie...35

2..Charlie...35

Activate Windows
Go to Settings to activate Windows.